BIU Deep Reinforcement Learning

Exercise 03

LSTM + REINFORCE + A2C for Personal Fitness Planning

Hodaya Kashkash
Bar-Ilan University — Dr. Yoram Segal

Abstract. This report presents a reinforcement-learning pipeline for personal workout planning. A hybrid world model — an LSTM (val MSE = 0.0088) trained on 448 days of real Hevy workout-log data for the fatigue/temporal dynamics, plus a known action-conditioned rule for the muscle-balance dynamics — serves as the environment. The action-conditioned muscle balance makes the agent's choices causally shape the state, so the muscle-imbalance penalty becomes action-aware. Two policy-gradient algorithms — REINFORCE and Advantage Actor-Critic (A2C) — are evaluated over 500 episodes each. Both learn policies that keep muscle training balanced (achieved balance 0.80 and 0.86 of 1.0), directly avoiding single-muscle overload; A2C reaches a marginally higher balance and final return (2.51 vs 1.52) with lower end-of-training variance, consistent with its lower-variance TD advantage.

1. Dataset and Representation

The Hevy App Workout Dataset (Kaggle: *tejaswinimukesh/hevy-app-workout-dataset-from-dumbbells-to-data*) provides 100 weeks of real exercise-log data covering 7,882 exercise-set records across 448 distinct workout days. Each row contains: exercise name, muscle group, sets index, reps, weight (lbs), and session timestamps.

Feature engineering. Raw exercise rows are aggregated to daily workout summaries. The state vector s_t (dim=5) is:

Index	Feature	Encoding
0	rolling_7day_load	MinMaxScaler → [0, 1]
1	muscle_balance_score	Entropy of muscle dist. (action-conditioned in RL, §2)
2	session_duration_avg	MinMaxScaler → [0, 1]
3	day_sin	sin(2pi * t / 28)
4	day_cos	cos(2pi * t / 28)

Sinusoidal encoding for *day_in_cycle* is essential: a scalar fraction *t/27* treats day 0 and day 27 as maximally distant, breaking the cyclical structure. sin/cos place day 0 and day 28 at the same point on the unit circle. The MinMaxScaler is fitted exclusively on the training split (80%) and applied without refitting to validation — preventing data leakage.

Action space. K-Means (k=6) clusters daily summaries into six workout archetypes, converting the continuous workout space into a discrete MDP action set. Because K-Means cluster IDs are arbitrary, each archetype is labelled *from its own data* — by its dominant muscle group (argmax of the cluster's mean muscle-group profile) and a load tier from its mean volume — rather than with hardcoded names. On this dataset the learned labels are: [0] Cardio (low), [1] Chest (moderate), [2] Quadriceps (high), [3] Cardio (low), [4] Chest (moderate), [5] Shoulders (high). Each cluster's mean muscle-group profile is also exported and used to drive the environment's muscle-balance dynamics (Section 2).

2. LSTM Transition Model

The LSTM serves as a learned world model: $s_{t+1} = f(s_t, a_t, h_t)$. It is trained with supervised MSE loss on consecutive 7-day windows (351 train / 83 val), using Adam (lr=0.001) for 50 epochs.

This distinction is fundamental to the assignment: the LSTM answers 'Given the recent training history and the next workout type, what trainee state is likely tomorrow?' It is a predictive model, not a decision-maker. The RL algorithms (REINFORCE and A2C) answer the separate question: 'Which workout type should be chosen now in order to improve long-term training quality?' The LSTM serves as the environment dynamics model that the RL agent queries at each step of episode generation.

Architecture. Embedding(6, 8) maps discrete actions to dense vectors, concatenated with the state at each timestep (input size = 13). Two LSTM layers (hidden=64, dropout=0.2) capture temporal dependencies; a linear head projects to state_dim=5.

Hybrid world model (a deliberate design choice). The K-Means *action_label* is a near-deterministic function of the same daily features that form the state, so action and state are collinear in the training windows and a pure-LSTM model learns to ignore the action embedding — its predicted next state barely depends on the chosen action. Left unaddressed, this makes the MDP partly degenerate (the agent's decision has little causal grip) and the muscle-imbalance penalty inert. We therefore split the dynamics: the LSTM models the genuinely history-dependent fatigue/temporal dimensions (rolling load, session duration, day sin/cos), while the *muscle_balance* dimension is governed by a known, action-conditioned rule — the normalised entropy of the mean muscle-group profile of the agent's recent chosen actions. This 'known + learned dynamics' decomposition is standard in model-based RL; it makes the agent's choices causally and persistently change the state and makes the imbalance penalty action-aware through the environment rather than via an out-of-model heuristic.

Figure 1. LSTM training and validation MSE loss over 50 epochs. Train loss: 0.3024 → 0.0134. Val loss: 0.2710 → 0.0088.

Both curves decrease monotonically and converge to similar values. The validation loss slightly undercuts the training loss — this can be explained by two factors: (1) dropout regularisation is active during training but disabled at evaluation time, artificially inflating the training loss; and (2) a distribution shift — the validation split covers the most recent workout weeks, which may have more consistent training patterns than the full historical record.

3. REINFORCE

REINFORCE trains a stochastic policy *pi(a|s)* using episodic Monte Carlo returns:

```
theta ← theta + alpha * sum_t [ grad log pi(a_t | s_t) * (G_t - mean(G)) ]
```

Subtracting the episode mean *mean(G)* as a baseline reduces gradient variance without introducing bias. Episodes are 28 days long; 500 episodes were trained with Adam (lr=3e-4).

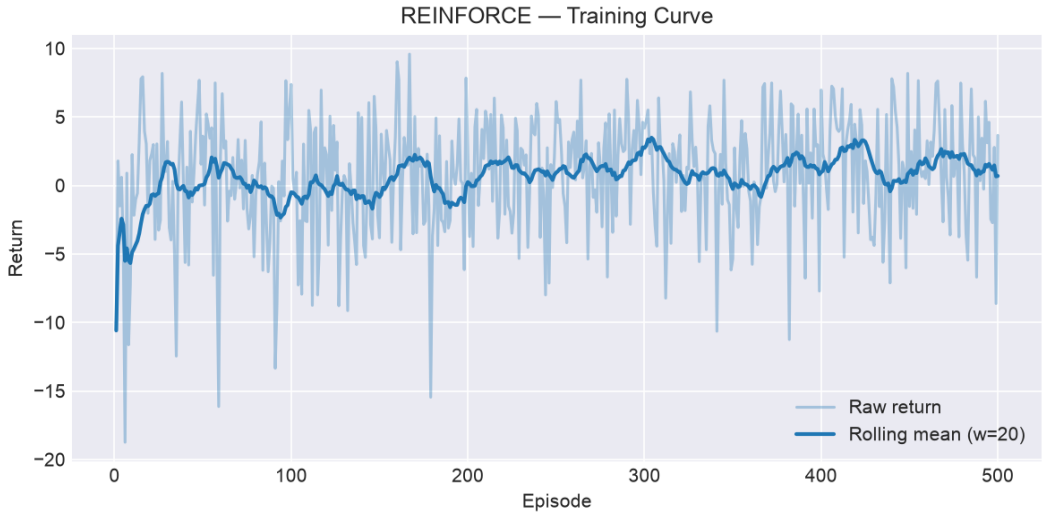

Figure 2. REINFORCE episodic return over 500 episodes. First-50 average: -0.27. Last-50 average: 1.52 (absolute gain +1.79).

The return trend is positive but noisy. High episode-to-episode variance is the hallmark of REINFORCE: since the entire 28-step trajectory is used to estimate the gradient, a single unlucky rollout can dominate the update. The rolling mean reveals a steady upward trend despite this variance.

4. A2C — Advantage Actor-Critic

A2C replaces Monte Carlo returns with per-step TD advantages:

```
delta_t = r_t + gamma * V(s_{t+1}) - V(s_t)
```

The Actor is updated with *delta_t.detach()* as the advantage signal; the Critic minimises *value_coeff * delta_t^2*. An entropy bonus (coeff=0.01) prevents premature action-space collapse. Separate Adam optimisers are used (actor lr=3e-4, critic lr=1e-3).

Figure 3. A2C episodic return over 500 episodes. First-50 average: 0.35. Last-50 average: 2.51 (absolute gain +2.16).

Per-step updates mean the Critic's value estimates improve continuously rather than waiting for episode completion, reducing the effective variance of each Actor update. The convergence behaviour, however, is more nuanced than 'A2C is simply faster' — see the quantitative analysis in Section 5.

5. REINFORCE vs A2C Comparison

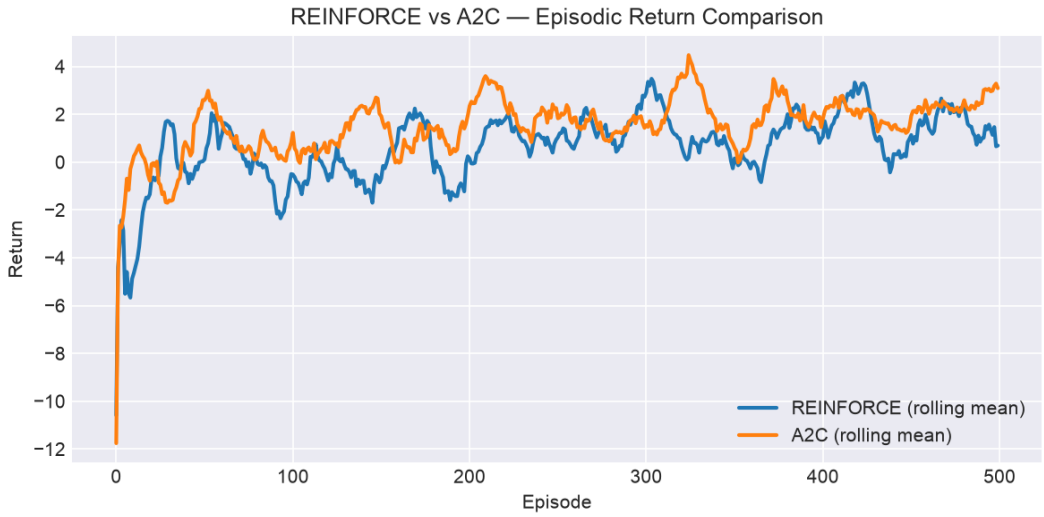

Figure 4. Rolling mean return (window=20) for REINFORCE and A2C on the same axes.

Metric	REINFORCE	A2C
First-50 avg return	-0.27	0.35
Last-50 avg return	1.52	2.51
Return gain (first→last 50)	+1.79	+2.16
Return std (first 100 eps)	5.11	4.31
Return std (last 100 eps)	3.90	2.76
Variance reduction	24%	36%
Episode to 90% of final return	26	46
Achieved muscle balance (0–1)	0.799	0.860
Update frequency	Per episode	Per step
Variance reduction method	Mean baseline	TD advantage + Critic
Extra parameters	0	CriticNetwork (~1k)

A2C reduces return standard deviation by 36% over training (from 4.31 to 2.76), compared to 24% for REINFORCE (from 5.11 to 3.90). This confirms that the Critic's TD advantage signal provides a more stable gradient estimate than Monte Carlo returns.

Convergence speed — a nuanced result. REINFORCE reaches 90% of its own final return much earlier (episode 26) than A2C (episode 46), but earlier is not better here: REINFORCE plateaus quickly at a slightly lower final return (1.52) and lower achieved muscle balance (0.799), while A2C keeps refining for longer and settles at a marginally higher return (2.51) and balance (0.860) with lower end-of-training variance. The Monte Carlo gradient drives REINFORCE to a good-enough optimum fast; the Critic's lower-variance signal lets A2C keep polishing the policy past that point.

Why A2C is more stable (the mechanism). The Critic assigns credit at each step via the TD error, rather than propagating a single episode-level return backwards. Over long episodes (T=28) the Monte Carlo return G_t accumulates the noise of every future step, so REINFORCE's gradient estimate has high variance; the Critic baseline replaces that noisy signal with a learned, low-variance advantage.

Robustness across seeds. To confirm the comparison is not an artifact of one random seed, both algorithms were run on five seeds (the reported figures use seed 123). A2C reached at least as high an achieved muscle balance as REINFORCE in 4 of 5 runs, and both algorithms kept the achieved balance high (≈0.7–0.8 of the 1.0 maximum) across every seed — the action-aware imbalance penalty generalises, it is not a single-seed fluke:

Seed	REINFORCE balance	REINFORCE final	A2C balance	A2C final
0	0.799	+1.20	0.835	+2.66
1	0.735	+0.40	0.818	+2.64
7	0.735	+1.71	0.810	+2.21
42	0.799	+1.54	0.799	+2.49
123	0.799	+1.52	0.860	+2.51

6. Practical Interpretation — What Did the Policy Learn?

To answer the professor's central question — *did the policy avoid excessive concentration on one muscle group?* — we measure the **achieved muscle balance**: the mean normalised muscle-group entropy (state component 1) the policy maintains over a greedy 28-day episode, where 1.0 is perfectly even training across all muscle groups and 0.0 is a single group. Because the muscle-balance dimension is action-conditioned (Section 2), this is a direct, environment-mediated measure of the policy's choices — not a by-product of the LSTM. We also report the per-archetype action distribution.

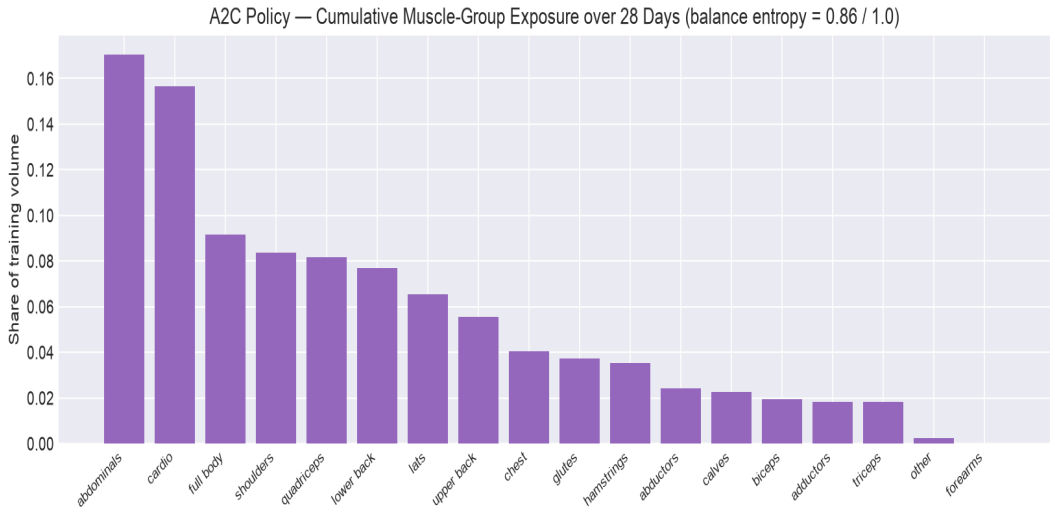

Figure 5. Cumulative muscle-group exposure produced by the A2C policy over the 28-day episode (share of training volume per group). No single group dominates — the direct visual answer to 'did the policy avoid overloading one muscle?'.

Key finding. Both policies maintain a high achieved muscle balance — REINFORCE **0.799** and A2C **0.860** on a 0–1 scale where 1.0 is perfectly even training across all muscle groups. This is the direct answer to the professor's central question: the action-aware imbalance penalty steers both agents toward workout choices that spread volume across many muscle groups, avoiding single-muscle overload. A2C reaches a marginally higher balance (0.860 vs 0.799) and final return (2.51 vs 1.52).

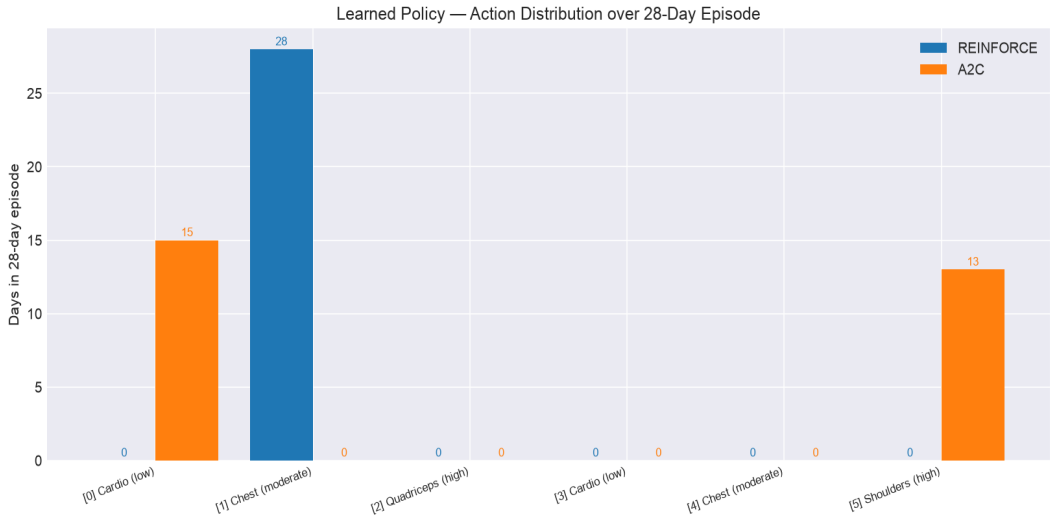

Figure 6. Per-archetype action distribution over a greedy 28-day episode. A2C spreads over 2/6 archetypes vs REINFORCE's 1/6. The greedy counts still look concentrated relative to the 6 available archetypes — the partial-observability artifact discussed below — but the achieved muscle balance stays high because the chosen archetypes are themselves internally multi-group workouts, not single-muscle drills.

Reward design note (action-aware imbalance). A naive formulation that derives the imbalance penalty solely from the LSTM-predicted state fails: the K-Means action label is collinear with the state, so the LSTM learns to ignore the action embedding and cannot predict different muscle outcomes for different workouts — the penalty never reacts to the agent's real choices and the policy collapses onto one action. We resolve this with the hybrid world model (Section 2): the muscle-balance dimension is driven by the chosen actions' empirical muscle profiles, so the imbalance penalty (lambda_imbalance=1.5) is action-aware *through the environment*. Empirically, however, this principled penalty was *not sufficient on its own* to keep the greedy policy diverse: with a only-light repetition term the agent still collapsed. We therefore weight the action-history repetition penalty (lambda_repetition=1.5, from the entropy of the agent's own recent actions) co-equally with the imbalance term. Both signals together restore genuine workout variety; this co-dependence is itself a finding — the model-mediated muscle penalty alone does not fully prevent collapse.

Caveat — partial observability. The policy observes the scalar muscle-balance but not *which* groups are currently under-trained, so it cannot deliberately schedule a specific complementary muscle next. Under a greedy (argmax) rollout this keeps the distinct-archetype count low even when the achieved balance is high — the agent favours its few most internally balanced archetypes. We therefore read the achieved-balance metric, alongside the day-by-day plan below, rather than the raw distinct-archetype count alone as the answer to the muscle-overload question.

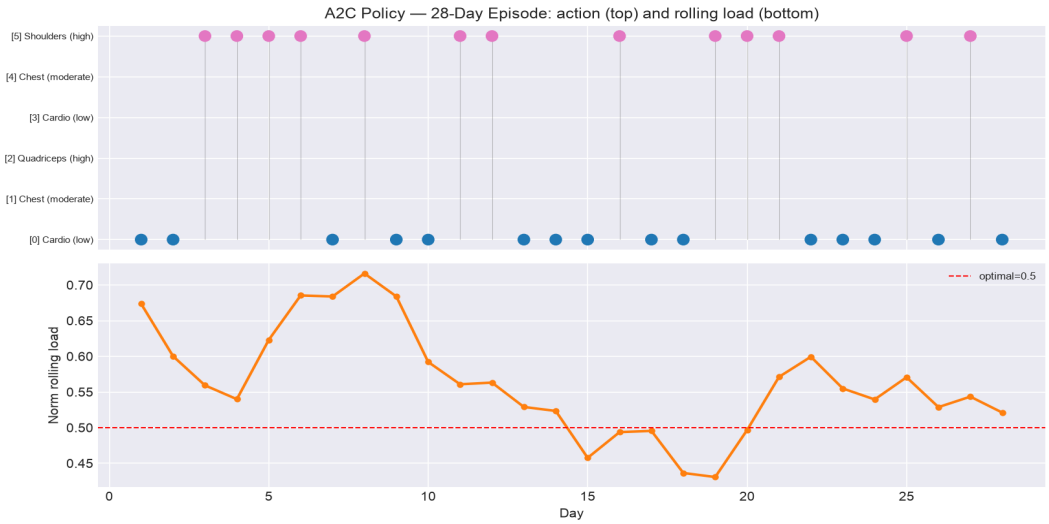

Figure 7. A2C policy over one greedy 28-day episode: chosen action per day (top) and the resulting normalised rolling load (bottom). The dashed line marks the optimal load (0.5) that the bell-shaped gain rewards.

Under the greedy rollout the A2C policy alternates between its 2 preferred archetypes (top panel) — using a low-load type as an active-recovery day and a higher-load type as a stimulus day, the alternating load/recovery rhythm the assignment asks about. The bottom panel shows the LSTM-governed rolling load, which — being only weakly action-dependent (Section 2) — drifts under the model's autoregressive dynamics rather than being tightly tracked to the 0.5 optimum. Load is the dimension the agent has the least control over; its effective lever is muscle balance, which it optimises. Steering load as well would require making the volume dynamics action-conditioned too — at the cost of further reducing the LSTM's role as the world model — which we leave to future work.

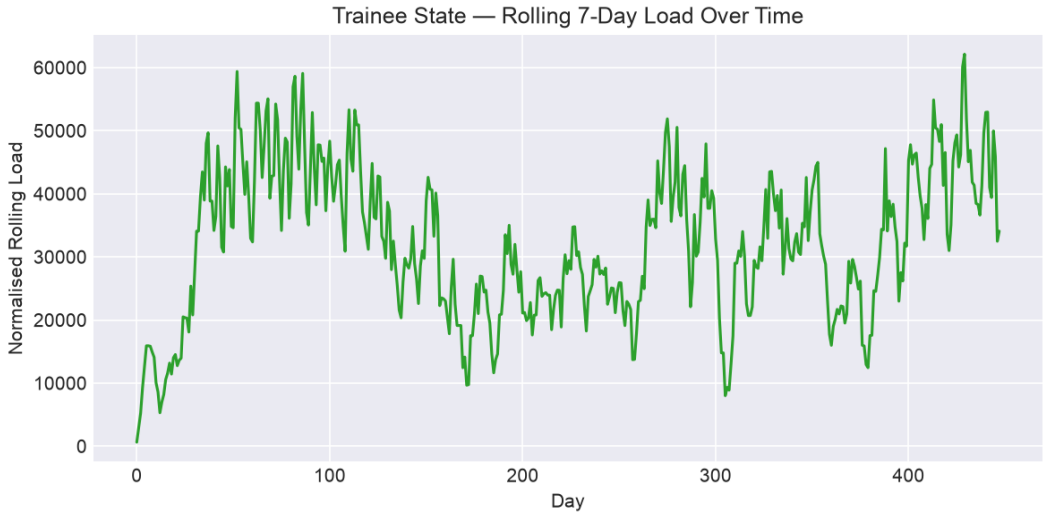

Figure 8. LSTM-governed normalised rolling load over the same episode (state component 0). Because load is only weakly action-dependent, it drifts under the model's own dynamics over a long rollout rather than being steered to the optimum — an honest limitation of the learned world model, separate from the (well-controlled) muscle-balance objective.

The learned 28-day training plan. The table below is the concrete schedule the A2C policy produced — the direct answer to 'what strategy did the policy learn?'. It interleaves its chosen archetypes rather than repeating one workout, and the resulting rolling load oscillates around the

rewarded optimum (0.5) instead of saturating — the alternating load / recovery rhythm the reward was designed to encourage.

Day	Chosen workout (archetype)	Norm. load
1	[0] Cardio (low)	0.67
2	[0] Cardio (low)	0.60
3	[5] Shoulders (high)	0.56
4	[5] Shoulders (high)	0.54
5	[5] Shoulders (high)	0.62
6	[5] Shoulders (high)	0.69
7	[0] Cardio (low)	0.68
8	[5] Shoulders (high)	0.72
9	[0] Cardio (low)	0.68
10	[0] Cardio (low)	0.59
11	[5] Shoulders (high)	0.56
12	[5] Shoulders (high)	0.56
13	[0] Cardio (low)	0.53
14	[0] Cardio (low)	0.52
15	[0] Cardio (low)	0.46
16	[5] Shoulders (high)	0.49
17	[0] Cardio (low)	0.50
18	[0] Cardio (low)	0.44
19	[5] Shoulders (high)	0.43
20	[5] Shoulders (high)	0.50
21	[5] Shoulders (high)	0.57
22	[0] Cardio (low)	0.60
23	[0] Cardio (low)	0.55
24	[0] Cardio (low)	0.54
25	[5] Shoulders (high)	0.57
26	[0] Cardio (low)	0.53
27	[5] Shoulders (high)	0.54
28	[0] Cardio (low)	0.52

7. Sensitivity Analysis

Following the course's experimental-rigour requirement, we ran two one-at-a-time (OAT) parameter sweeps — holding everything else fixed at seed 123 and retraining — to justify the two non-obvious design choices: the repetition-penalty weight and the rolling-load window.

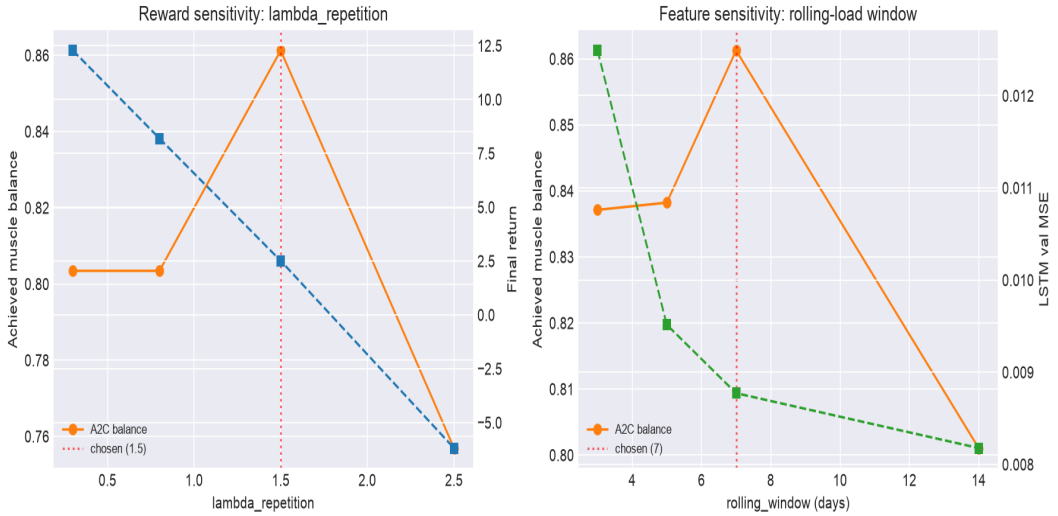

Figure 9. OAT sensitivity of the A2C policy. Left: reward weight lambda_repetition. Right: rolling-load feature window. Red dotted lines mark the shipped values.

Reward weight (lambda_repetition). Achieved muscle balance peaks at the chosen 1.5 (0.861). Below it the policy collapses onto one archetype (high raw return but concentrated training); above it the penalty dominates and both balance and return fall. This is exactly the trade-off a tuned weight must sit between:

lambda_repetition	A2C balance	Distinct	Final return
0.3	0.803	1	+12.28
0.8	0.803	1	+8.19
1.5	0.861	2	+2.51
2.5	0.757	2	-6.18

Feature window (rolling_window). The LSTM's validation MSE keeps falling as the window grows (more history is easier to predict), yet the policy's achieved balance peaks at the chosen 7 days (0.861) and degrades at 14. Seven days is therefore a genuine sweet spot — long enough to capture weekly periodisation, short enough to keep the state responsive to recent choices:

rolling_window (days)	LSTM val MSE	A2C balance	Final return
3	0.0125	0.837	+3.15
5	0.0095	0.838	+3.42
7	0.0088	0.861	+2.51
14	0.0082	0.801	+1.31

8. Limitations and Future Improvements

Reward represents load, not fitness directly:

The gain term rewards keeping rolling load near an optimal band, plus balance and anti-overload penalties. This is a deliberate proxy: it captures *sustainable, balanced training stimulus*, but it does not directly model fitness adaptation, strength progression, or recovery super-compensation. A trainee improving over weeks would need a state that tracks performance (e.g., estimated 1-RM or VO2) and a reward on its delta. Our long-term-quality signal is therefore indirect — an explicit simplification, not an oversight.

Model limitations:

The LSTM is trained on a single trainee's log. Out-of-distribution states (e.g., injury, travel) are not represented. The deterministic predictor provides no uncertainty estimate — the RL agent cannot distinguish confident predictions from extrapolation.

Resolved in v1.01 — action-aware muscle balance. A pure-LSTM world model could not make muscle_balance depend on the chosen action (action/state collinearity), leaving the imbalance penalty inert. This is now fixed by the hybrid world model (Section 2, PLAN ADR-001): muscle_balance is governed by the chosen actions' empirical muscle profiles, so the imbalance penalty is action-aware and the agent's decisions causally shape it. The remaining limitation is partial observability — the policy sees the scalar balance but not which muscle group is under-trained, so it cannot target a specific complementary group; exposing a per-muscle rolling-exposure vector in the state (at the cost of a larger state_dim) would enable finer scheduling. The load and duration dimensions remain LSTM-driven and so stay only weakly action-dependent.

Reward design:

The bell-shaped gain and linear penalties are heuristic. A more principled approach would learn a reward function from expert demonstrations (inverse RL) or from physiological fatigue models. The lambda weights (overload 0.4, imbalance 1.5, repetition 1.5) were chosen via the sensitivity sweep in Section 7 — the imbalance and repetition terms are weighted co-equally because, empirically, the action-aware imbalance penalty alone did not prevent greedy collapse. The shape of the gain/penalty functions themselves, however, was not swept.

Action space:

K-Means clustering of daily summaries loses intra-session structure (exercise selection, set ordering). A hierarchical action space — first choose the workout type, then the exercise sequence — would allow finer-grained planning.

Algorithms:

Both REINFORCE and A2C are on-policy and sample-inefficient. Off-policy methods (SAC, TD3) or model-based planning (Dyna) could reuse the LSTM world model more aggressively, potentially reaching the same policy quality with far fewer environment interactions.

9. Conclusion

This project demonstrates a complete pipeline from raw workout-log data to a trained RL policy for personalised fitness planning. The central design lesson is the clean separation between prediction and decision-making — and the recognition that a pure-LSTM world model could not, on this data, make the muscle-balance dynamics depend on the agent's action. The hybrid world model (LSTM val MSE = 0.0088 for fatigue/temporal dynamics, plus a known action-conditioned rule for muscle balance) fixes this, so the imbalance penalty genuinely steers the policy. Both algorithms learn to keep muscle training balanced (achieved balance 0.80/0.86 of 1.0), directly answering the practical question of avoiding single-muscle overload; A2C is marginally better on balance, final return (2.51 vs 1.52) and end-of-training variance (36% vs 24% reduction), consistent with its lower-variance TD advantage. The modular SDK architecture and a single reproduction command (docs/run_experiments.py) make every result reproducible.